

Project: Dinamica d'impresa, SDE e Fokker–Planck

dinamiche stocastiche, distribuzioni stazionarie e inferenza
economica

1. Obiettivi della dispensa

Questa dispensa introduce un modello stocastico molto semplice ma molto ricco per descrivere la dinamica di una variabile d'impresa, per esempio dimensione, log-dimensione o tasso di crescita cumulato, mediante una equazione differenziale stocastica unidimensionale.

Gli obiettivi sono cinque:

1. formalizzare una dinamica continua con drift e diffusione;
2. derivare la distribuzione stazionaria mediante l'equazione di Fokker–Planck;
3. mostrare come una famiglia di distribuzioni target implichi un drift coerente;
4. discutere il caso Subbotin e il caso Laplace;
5. chiarire il limite economico del modello quando entry ed exit non sono trattate esplicitamente.

Dal punto di vista del corso, questo e' un caso di studio particolarmente utile perche' si colloca esattamente tra economia, probabilita' applicata e inferenza statistica.

2. Motivazione generale

Molti fenomeni economici possono essere descritti, in prima approssimazione, come dinamiche casuali con due ingredienti fondamentali:

- una componente sistematica che tende a spingere la variabile in una certa direzione;
- una componente casuale che rappresenta shock, incertezza, eterogeneita' non osservata o fluttuazioni idiosincratiche.

Nel caso delle imprese, una variabile naturale e' una misura di dimensione, produttivita', quota di mercato o log-dimensione. Se questa variabile evolve nel

tempo sotto l'effetto congiunto di drift e rumore, allora una descrizione continua tramite SDE e' spesso un primo modello molto naturale.

Questa impostazione e' utile non solo per simulare traiettorie, ma anche per collegare la dinamica microscopica alla distribuzione osservata in sezione trasversale.

3. Il modello stocastico di base

Consideriamo la seguente equazione differenziale stocastica unidimensionale:

$$dX_t = A(X_t) dt + \sqrt{D(X_t)} dW_t.$$

Qui:

- X_t e' la variabile d'impresa al tempo t ;
- $A(X_t)$ e' il drift;
- $D(X_t)$ e' la funzione di diffusione;
- W_t e' un moto browniano standard.

Interpretazione economica:

- il drift $A(x)$ descrive la tendenza media del sistema quando la variabile si trova nel punto x ;
- la diffusione $D(x)$ descrive l'intensita' delle fluttuazioni casuali attorno a quella tendenza.

Se $A(x)$ e' negativo per valori grandi di x e positivo per valori piccoli, il sistema ha una tendenza di mean reversion. Se invece il drift e' quasi nullo, la dinamica puo' essere dominata dagli shock.

4. Dalla SDE alla Fokker–Planck

La controparte in termini di densita' di probabilita' del processo e' l'equazione di Fokker–Planck associata. Essa descrive l'evoluzione temporale della densita' $p(x, t)$ della variabile X_t .

Nel caso unidimensionale, il punto chiave per questa dispensa non e' tanto l'equazione completa dipendente dal tempo, quanto la sua soluzione stazionaria. La nota scrive infatti direttamente una forma generale della densita' di equilibrio:

$$p_{\text{eq}}(n) = \frac{k}{D(n)} \exp \left(2 \int_{n_0}^n \frac{A(y)}{D(y)} dy \right),$$

dove:

- k e' una costante di normalizzazione;
- n_0 e' un punto di riferimento;
- A e D sono rispettivamente drift e diffusione.

Questa formula e' molto importante, perche' collega direttamente il modello dinamico alla distribuzione stazionaria osservabile.

5. Il problema inverso

Una delle idee piu' interessanti della nota e' il seguente ragionamento inverso.

Invece di partire da un drift e da una diffusione e poi calcolare la distribuzione stazionaria, si puo' fare il contrario:

1. si ipotizza una forma plausibile della distribuzione stazionaria;
2. si usa la formula di equilibrio per ricavare il drift compatibile con quella distribuzione.

Questo e' molto utile quando i dati empirici suggeriscono una famiglia di distribuzioni ben identificabile.

6. Caso a diffusione costante

Supponiamo ora che la diffusione sia costante:

$$D(n) = D.$$

In questo caso la formula stazionaria si semplifica. Derivando il logaritmo della densita' si ottiene

$$\partial_n \ln p_{\text{eq}}(n) = \frac{2A(n)}{D}.$$

Questa relazione e' estremamente utile: se si specifica $p_{\text{eq}}(n)$, allora si puo' ricavare immediatamente il drift $A(n)$.

In altre parole, la derivata logaritmica della densita' stazionaria determina il campo di drift.

7. Distribuzione Subbotin

Supponiamo ora che la densita' di equilibrio abbia forma Subbotin:

$$p_{\text{eq}}(n) \propto \exp \left[-\frac{1}{\alpha} \left| \frac{n-m}{\sigma} \right|^\alpha \right].$$

Qui:

- m e' un parametro di posizione;
- σ e' un parametro di scala;
- α controlla la forma della coda e del picco centrale.

Questa famiglia e' molto flessibile:

- per $\alpha = 2$ si ottiene una forma gaussiana;
- per $\alpha = 1$ si ottiene la Laplace;
- per altri valori di α si ottengono profili piu' appuntiti o piu' smussati.

Usando la relazione tra densita' stazionaria e drift, la nota ricava

$$A(n) = -\frac{D}{2\sigma} \operatorname{sign}(n - m) \left| \frac{n - m}{\sigma} \right|^{\alpha-1}.$$

Quindi il drift e' diretto verso il centro m e la sua intensita' dipende dalla distanza da m e dal parametro α .

8. La SDE compatibile con la Subbotin

Sostituendo questo drift nella SDE si ottiene

$$dX_t = -\frac{D}{2\sigma} \operatorname{sign}(X_t - m) \left| \frac{X_t - m}{\sigma} \right|^{\alpha-1} dt + D^{1/2} dW_t.$$

Questa equazione ha, per costruzione, una distribuzione stazionaria di tipo Subbotin.

Dal punto di vista concettuale, questa e' una costruzione molto elegante:

- si sceglie la forma stazionaria desiderata;
- si deduce il drift necessario per sostenerla;
- si ottiene una dinamica continua coerente con la distribuzione empirica.

9. Caso Laplace

Il caso $\alpha = 1$ corrisponde alla distribuzione di Laplace. In questo caso il drift si semplifica molto e diventa

$$A(n) = -\frac{D}{2\sigma} \operatorname{sign}(n - m).$$

La SDE corrispondente e' quindi

$$dX_t = -\frac{D}{2\sigma} \operatorname{sign}(X_t - m) dt + D^{1/2} dW_t.$$

Questo significa che il processo subisce una forza di richiamo verso m con intensita' costante in modulo, piuttosto che proporzionale alla distanza.

Il caso Laplace e' particolarmente interessante dal punto di vista empirico, perche' distribuzioni simili compaiono spesso in dati economici con code piu' pesanti di quelle gaussiane.

10. Interpretazione economica del drift

Il drift ricostruito dalla distribuzione stazionaria puo' essere interpretato economicamente come una forza di rientro verso una dimensione tipica o una regione centrale dello spazio degli stati.

Questa lettura va trattata con cautela, ma e' molto utile:

- se il drift e' forte, la variabile tende a rientrare rapidamente;
- se il rumore e' forte, le traiettorie restano molto disperse;
- la distribuzione osservata e' il risultato dell'equilibrio tra richiamo sistematico e fluttuazioni casuali.

Nel caso Subbotin, la forma del drift cambia con α :

- per $\alpha > 1$ il richiamo cresce con la distanza;
- per $\alpha = 1$ il richiamo ha modulo costante;
- per $\alpha < 1$ si ottengono forme ancora piu' singolari vicino al centro.

11. Cosa spiega il modello e cosa non spiega

Il modello spiega bene una cosa precisa: come una dinamica continua con drift e diffusione possa sostenere una certa distribuzione stazionaria osservata.

Ma la nota segnala anche un limite molto importante: questo tipo di approccio adatta distribuzioni di equilibrio a dati cross-sectional senza rappresentare esplicitamente la dinamica di ingresso e uscita delle imprese.

Questo punto e' cruciale.

Se si osserva la distribuzione delle dimensioni delle imprese in una economia reale, tale distribuzione non dipende solo dalla crescita delle imprese gia' esistenti, ma anche da:

- nascite di nuove imprese;
- fallimenti;
- fusioni;
- uscita dal mercato;
- selezione.

Quindi un modello continuo di sola crescita individuale puo' essere molto utile, ma resta incompleto se non si include anche la demografia delle imprese.

12. Perche' questo e' un buon case study per il corso

Questa dispensa e' molto adatta a un corso di metodi computazionali per almeno quattro ragioni.

Primo, il modello e' matematicamente compatto ma concettualmente molto ricco.

Secondo, permette di lavorare sia in avanti sia all'indietro:

- dalla SDE alla distribuzione;
- dalla distribuzione alla SDE.

Terzo, si presta molto bene a diversi metodi computazionali:

- simulazione Euler–Maruyama;
- stima empirica della distribuzione stazionaria;
- confronto tra istogrammi simulati e densita' teoriche;
- discretizzazione numerica dell'equazione di Fokker–Planck.

Quarto, apre immediatamente la strada a una estensione economicamente molto naturale: aggiungere birth e death di imprese.

13. Simulazione numerica: Euler–Maruyama

La simulazione di una SDE del tipo

$$dX_t = A(X_t) dt + \sqrt{D(X_t)} dW_t$$

si puo' fare con lo schema di Euler–Maruyama:

$$X_{t+\Delta t} = X_t + A(X_t)\Delta t + \sqrt{D(X_t)}\sqrt{\Delta t}\xi_t,$$

dove

$$\xi_t \sim \mathcal{N}(0, 1).$$

Questo schema e' il modo piu' semplice per passare dalla formulazione continua alla simulazione di traiettorie discrete.

14. Stima empirica della distribuzione stazionaria

Una volta simulata la dinamica per tempi lunghi, si puo' stimare la distribuzione stazionaria empirica in modo molto semplice:

1. si genera una traiettoria lunga;
2. si scarta un transiente iniziale;
3. si costruisce un istogramma dei valori osservati;
4. si confronta l'istogramma con la densita' teorica.

Questo e' didatticamente molto utile, perche' rende visibile il collegamento tra dinamica temporale e distribuzione di equilibrio.

15. Soluzione numerica della Fokker–Planck

Un secondo approccio computazionale consiste nel non simulare direttamente le traiettorie, ma discretizzare l'equazione di Fokker–Planck per la densita' $p(x, t)$.

In questo caso l'oggetto evoluto non e' la traiettoria di una singola impresa, ma la distribuzione dell'intera popolazione di imprese.

Dal punto di vista del corso, questo confronto e' importante:

- simulare la SDE significa lavorare a livello di traiettorie;
- risolvere la Fokker–Planck significa lavorare a livello di densita'.

Questa distinzione e' una delle piu' istruttive dell'intero modulo.

16. Pseudocodice per la simulazione Euler–Maruyama

Input

- drift $A(x)$
- diffusione $D(x)$
- valore iniziale X_0
- passo temporale Δt
- numero di iterazioni T

Pseudocodice

1. inizializza $X = X_0$
2. per $t = 1, \dots, T$:
 - estrai

$$\xi_t \sim \mathcal{N}(0, 1)$$

- aggiorna

$$X \leftarrow X + A(X)\Delta t + \sqrt{D(X)}\sqrt{\Delta t}\xi_t$$

- salva il valore

3. restituisci la traiettoria

Questo e' il punto di partenza naturale per il laboratorio.

17. Pseudocodice per la stima della stazionaria

1. simula una traiettoria lunga;
2. scarta i primi passi come burn-in;
3. raccogli i valori restanti;
4. costruisci un istogramma;
5. confronta con la densita' teorica.

Questo esperimento numerico permette di verificare concretamente se la SDE simulata produce la distribuzione prevista dalla teoria.

18. Possibile estensione: nascita e morte delle imprese

Una estensione molto naturale consiste nell'aggiungere una dinamica di demografia d'impresa. Per esempio:

- con una certa probabilit  nasce una nuova impresa, con stato iniziale estratto da una distribuzione di entrata;
- con una certa probabilit  una impresa esce dal mercato, magari se la sua variabile cade sotto una soglia.

In questo modo il sistema non   pi  una semplice SDE per una singola impresa, ma un processo stocastico per una popolazione mutevole di imprese.

Questa estensione   particolarmente importante dal punto di vista economico, perche  corregge esattamente il limite segnalato nella nota.

19. Schema del laboratorio

19.1 Laboratorio 1 - Simulazione di traiettorie

Obiettivo

Simulare una SDE con drift e diffusione dati.

Attivit 

1. implementare Euler-Maruyama;
2. scegliere parametri semplici;
3. generare diverse traiettorie;
4. osservare il ruolo di drift e diffusione.

Domande guida

- che effetto ha aumentare D ?
- il drift riporta la traiettoria verso il centro?
- il comportamento cambia molto tra Subbotin e Laplace?

19.2 Laboratorio 2 - Distribuzione stazionaria empirica

Obiettivo

Verificare numericamente la forma della distribuzione stazionaria.

Attivit 

1. simulare una traiettoria lunga;
2. scartare il transiente;
3. costruire un istogramma;
4. confrontare con la densit  teorica.

Domande guida

- l'istogramma converge alla distribuzione attesa?
- quanto deve essere lungo il burn-in?
- la stima cambia molto al variare del passo temporale?

19.3 Laboratorio 3 - Confronto tra traiettorie e densita'

Obiettivo

Confrontare simulazione della SDE e discretizzazione della Fokker–Planck.

Attivita'

1. simulare molte traiettorie;
2. stimare la distribuzione empirica;
3. risolvere numericamente la Fokker–Planck;
4. confrontare le due distribuzioni.

Domande guida

- i due approcci coincidono numericamente?
- quale e' piu' costoso?
- quale e' piu' naturale in un problema economico?

19.4 Laboratorio 4 - Birth e death

Obiettivo

Studiare l'effetto di ingresso e uscita delle imprese.

Attivita'

1. aggiungere una regola di nascita;
2. aggiungere una regola di morte;
3. confrontare la distribuzione finale con il caso senza demografia;
4. discutere l'effetto economico della selezione.

Domande guida

- la distribuzione cambia in modo sostanziale?
- il modello senza entry/exit e' ancora una buona approssimazione?
- quali nuove osservabili diventano rilevanti?

20. Conclusione

La dinamica d'impresa via SDE e Fokker–Planck e' un eccellente case study per un corso interdisciplinare, perche' unisce in modo molto pulito:

- modellizzazione stocastica continua;
- distribuzioni stazionarie;
- simulazione numerica;
- inferenza dalla distribuzione osservata;
- interpretazione economica dei limiti del modello.

Il punto concettuale piu' importante e' forse questo: osservare una distribuzione empirica non basta. Occorre chiedersi quale dinamica la sostenga e quali meccanismi, come entry ed exit, siano stati lasciati fuori dal modello.

21. Bibliografia minima

1. Risken, H. The Fokker–Planck Equation.
 2. Gardiner, C. W. Stochastic Methods.
 3. Sutton, J. Gibrat's Legacy.
 4. Hopenhayn, H. Entry, Exit, and Firm Dynamics in Long Run Equilibrium.
-

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python il modello di dinamica d'impresa basato su SDE e Fokker–Planck.

L'obiettivo non è costruire un codice sofisticato, ma fornire una guida che sia:

- leggibile da chi usa altri linguaggi, quindi vicina a uno pseudocodice operativo;
- quasi immediatamente implementabile da chi conosce Python.

Per questo motivo il codice è volutamente semplice:

- poche librerie;
- funzioni corte;
- passaggi espliciti;
- nessuna dipendenza avanzata.

La logica generale sarà questa:

1. definire drift e diffusione;
2. simulare la SDE con Euler–Maruyama;
3. stimare empiricamente la distribuzione stazionaria;
4. confrontarla con la densità teorica;
5. opzionalmente, discretizzare la Fokker–Planck;
6. aggiungere una estensione minimale con nascita e morte di imprese.

A.1 Librerie minime

Per questa appendice bastano:

```
import random
import math
import statistics
import matplotlib.pyplot as plt
```

Quindi:

- `random` serve per gli shock gaussiani e per gli eventi casuali;
- `math` serve per radici, esponenziali e funzione gamma;
- `statistics` serve per medie semplici;
- `matplotlib.pyplot` serve per grafici e istogrammi.

Non è necessario usare `numpy` in una prima implementazione.

A.2 Struttura generale del problema

Consideriamo una SDE unidimensionale della forma

$$dX_t = A(X_t)dt + \sqrt{D(X_t)}dW_t.$$

Nel codice conviene separare subito tre elementi:

1. la funzione di drift `drift(x, ...)`;
2. la funzione di diffusione `diffusion(x, ...)`;
3. la simulazione numerica della traiettoria.

A.3 Drift e diffusione

A.3.1 Diffusione costante

Il caso più semplice è una diffusione costante:

```
def constant_diffusion(x, D):
    return D
```

Qui `D` deve essere positiva.

A.3.2 Drift Subbotin

Per la distribuzione stazionaria di tipo Subbotin, con diffusione costante D , il drift della dispersione è

$$A(x) = -\frac{D}{2\sigma} \operatorname{sign}(x - m) \left| \frac{x - m}{\sigma} \right|^{\alpha-1}.$$

In Python:

```
def sign(value):
    if value > 0.0:
        return 1.0
    elif value < 0.0:
        return -1.0
    else:
        return 0.0

def subbotin_drift(x, m, sigma, alpha, D):
    z = (x - m) / sigma
    return -(D / (2.0 * sigma)) * sign(z) * (abs(z) ** (alpha - 1.0))
```

Questa funzione è il cuore del modello.

A.3.3 Drift Laplace

Il caso Laplace corrisponde a $\alpha = 1$, e il drift si riduce a

$$A(x) = -\frac{D}{2\sigma} \text{sign}(x - m).$$

Conviene comunque scriverlo anche in forma separata:

```
def laplace_drift(x, m, sigma, D):
    return -(D / (2.0 * sigma)) * sign(x - m)
```

A.3.4 Drift lineare gaussiano

Per confronto, può essere utile anche un drift lineare di tipo Ornstein–Uhlenbeck:

$$A(x) = -\kappa(x - m).$$

In Python:

```
def linear_mean_reverting_drift(x, m, kappa):
    return -kappa * (x - m)
```

Questa funzione non produce la Subbotin della dispensa, ma è utile come benchmark.

A.4 Un passo di Euler–Maruyama

Lo schema di Euler–Maruyama è

$$X_{t+\Delta t} = X_t + A(X_t)\Delta t + \sqrt{D(X_t)}\sqrt{\Delta t}\xi_t, \quad \xi_t \sim \mathcal{N}(0, 1).$$

In Python:

```
def euler_maruyama_step(x, dt, drift_function, diffusion_function):
    noise = random.gauss(0.0, 1.0)

    drift_value = drift_function(x)
    diffusion_value = diffusion_function(x)

    x_new = x + drift_value * dt + math.sqrt(diffusion_value * dt) * noise

    return x_new
```

Questa funzione è generica: può essere usata con qualsiasi drift e qualsiasi diffusione.

A.5 Simulare una traiettoria

Ora conviene costruire una funzione che simuli una traiettoria intera.

```
def simulate_sde(x0, dt, T, drift_function, diffusion_function):
    x = x0
    history = [x]

    for t in range(T):
        x = euler_maruyama_step(
            x=x,
            dt=dt,
            drift_function=drift_function,
            diffusion_function=diffusion_function
        )
        history.append(x)

    return history
```

Qui:

- x_0 è la condizione iniziale;
- dt è il passo temporale;
- T è il numero di iterazioni.

A.5.1 Esempio con drift Subbotin

```
m = 0.0
sigma = 1.0
alpha = 1.0
D = 0.5

def my_drift(x):
    return subbotin_drift(x, m=m, sigma=sigma, alpha=alpha, D=D)
```

```
def my_diffusion(x):
    return constant_diffusion(x, D=D)

history = simulate_sde(
    x0=4.0,
    dt=0.01,
    T=50000,
    drift_function=my_drift,
    diffusion_function=my_diffusion
)
```

Se $\alpha = 1.0$, questo è già il caso Laplace.

A.6 Grafico della traiettoria

```
def plot_trajectory(history, title="Traiettorie SDE"):
    times = list(range(len(history)))

    plt.plot(times, history)
    plt.xlabel("tempo discreto")
    plt.ylabel("X")
    plt.title(title)
    plt.show()
```

Esempio:

```
plot_trajectory(history, title="Traiettorie con drift Laplace")
```

Questo grafico serve a far vedere:

- il ruolo del drift;
- il ruolo del rumore;
- la presenza di mean reversion;
- la dipendenza dalla diffusione.

A.7 Raccogliere campioni stazionari

Per stimare la distribuzione stazionaria, conviene:

1. simulare una traiettoria lunga;
2. scartare un tratto iniziale di burn-in;
3. usare i valori restanti come campioni dalla distribuzione di equilibrio.

```
def stationary_samples_from_history(history, burn_in):
    return history[burn_in:]
```

Esempio:

```
samples = stationary_samples_from_history(history, burn_in=10000)
```

A.8 Istogramma empirico

```
def plot_stationary_histogram(samples, bins=60, title="Istogramma stazionario"):
    plt.hist(samples, bins=bins, density=True)
    plt.xlabel("x")
    plt.ylabel("densità empirica")
    plt.title(title)
    plt.show()
```

Esempio:

```
plot_stationary_histogram(samples, bins=80, title="Stazionaria empirica")
```

Questo è il primo confronto numerico essenziale.

A.9 Densità teorica Subbotin

La densità stazionaria della famiglia Subbotin nella parametrizzazione della dispensa è

$$p(x) = C \exp \left[-\frac{1}{\alpha} \left| \frac{x-m}{\sigma} \right|^\alpha \right],$$

con costante di normalizzazione

$$C = \frac{\alpha^{1-1/\alpha}}{2\sigma\Gamma(1/\alpha)}.$$

In Python:

```
def subbotin_density(x, m, sigma, alpha):
    coefficient = (alpha ** (1.0 - 1.0 / alpha)) / (2.0 * sigma * math.gamma(1.0 / alpha))
    exponent = -(1.0 / alpha) * (abs((x - m) / sigma) ** alpha)
    return coefficient * math.exp(exponent)
```

Nel caso $\alpha = 1$ questa formula restituisce la Laplace:

$$p(x) = \frac{1}{2\sigma} e^{-|x-m|/\sigma}.$$

A.10 Confronto tra istogramma e densità teorica

Per confrontare la stazionaria simulata con la densità attesa, conviene sovrapporre istogramma e curva teorica.

```

def plot_histogram_with_theoretical_density(samples, density_function,
                                           x_min=None, x_max=None,
                                           bins=60, num_points=400,
                                           title="Confronto con densità teorica"):

    if x_min is None:
        x_min = min(samples)
    if x_max is None:
        x_max = max(samples)

    plt.hist(samples, bins=bins, density=True, alpha=0.6)

    x_values = []
    y_values = []

    for n in range(num_points + 1):
        x = x_min + (x_max - x_min) * n / num_points
        y = density_function(x)
        x_values.append(x)
        y_values.append(y)

    plt.plot(x_values, y_values)
    plt.xlabel("x")
    plt.ylabel("densità")
    plt.title(title)
    plt.show()

```

Esempio:

```

def my_density(x):
    return subbotin_density(x, m=0.0, sigma=1.0, alpha=1.0)

plot_histogram_with_theoretical_density(
    samples=samples,
    density_function=my_density,
    bins=80,
    title="Laplace teorica vs stazionaria empirica"
)

```

A.11 Molte traiettorie indipendenti

Per ridurre la dipendenza da una singola traiettoria, si può ripetere la simulazione molte volte e raccogliere tutti i campioni finali o post burn-in.

```

def collect_stationary_samples_many_runs(num_runs, x0, dt, T, burn_in,
                                         drift_function, diffusion_function):

    all_samples = []

```



```

for run in range(num_runs):
    history = simulate_sde(
        x0=x0,
        dt=dt,
        T=T,
        drift_function=drift_function,
        diffusion_function=diffusion_function
    )

    samples = stationary_samples_from_history(history, burn_in=burn_in)

    for value in samples:
        all_samples.append(value)

return all_samples

```

Esempio:

```

many_samples = collect_stationary_samples_many_runs(
    num_runs=20,
    x0=4.0,
    dt=0.01,
    T=20000,
    burn_in=5000,
    drift_function=my_drift,
    diffusion_function=my_diffusion
)

```

Questo aiuta molto a costruire una stima empirica più robusta.

A.12 Medie e statistiche semplici

```

def sample_mean(values):
    return statistics.mean(values)

def sample_std(values):
    if len(values) > 1:
        return statistics.stdev(values)
    else:
        return 0.0

```

Esempio:

```

print("Media empirica:", sample_mean(many_samples))
print("Deviazione standard empirica:", sample_std(many_samples))

```

A.13 Solver molto semplice per la Fokker–Planck

Questa parte è opzionale, ma utile. L’idea è discretizzare l’equazione della densità su una griglia spaziale.

Per restare semplici, useremo una versione minimale del caso:

- diffusione costante D ;
- griglia uniforme;
- passo temporale piccolo;
- differenze finite esplicite.

L’equazione è

$$\partial_t p = -\partial_x(A(x)p) + \frac{D}{2}\partial_{xx}p.$$

A.13.1 Griglia iniziale

```
def create_grid(x_min, x_max, num_points):
    grid = []

    for n in range(num_points):
        x = x_min + (x_max - x_min) * n / (num_points - 1)
        grid.append(x)

    return grid
```

A.13.2 Densità iniziale

Per esempio, una densità iniziale localizzata attorno a un punto:

```
def initial_density_gaussian_like(grid, center, width):
    values = []

    for x in grid:
        value = math.exp(-((x - center) ** 2) / (2.0 * width ** 2))
        values.append(value)

    return normalize_density(values, grid)
```

A.13.3 Normalizzazione della densità

```
def normalize_density(p, grid):
    if len(grid) < 2:
        return p[:]
```

```

dx = grid[1] - grid[0]
total = sum(p) * dx

if total == 0.0:
    return p[:]

normalized = []
for value in p:
    normalized.append(value / total)

return normalized

```

A.13.4 Un passo esplicito di Fokker–Planck

```

def fokker_planck_step(grid, p, dt, drift_function, D):
    n = len(grid)
    dx = grid[1] - grid[0]

    new_p = p[:]

    for i in range(1, n - 1):
        x = grid[i]

        flux_right = drift_function(grid[i + 1]) * p[i + 1]
        flux_left = drift_function(grid[i - 1]) * p[i - 1]
        drift_term = -(flux_right - flux_left) / (2.0 * dx)

        diffusion_term = (D / 2.0) * (p[i + 1] - 2.0 * p[i] + p[i - 1]) / (dx ** 2)

        new_p[i] = p[i] + dt * (drift_term + diffusion_term)

    new_p[0] = 0.0
    new_p[-1] = 0.0

    new_p = normalize_density(new_p, grid)

    return new_p

```

Questa implementazione è volutamente elementare. Serve più come guida concettuale che come solver avanzato.

A.13.5 Simulazione completa della Fokker–Planck

```

def simulate_fokker_planck(grid, p0, dt, T, drift_function, D):
    p = p0[:]
    history = [p[:]]

```

```

for t in range(T):
    p = fokker_planck_step(
        grid=grid,
        p=p,
        dt=dt,
        drift_function=drift_function,
        D=D
    )
    history.append(p[:])

return history

```

A.13.6 Grafico della densità finale

```

def plot_density(grid, p, title="Densità"):
    plt.plot(grid, p)
    plt.xlabel("x")
    plt.ylabel("p(x)")
    plt.title(title)
    plt.show()

```

Esempio:

```

grid = create_grid(-8.0, 8.0, 400)
p0 = initial_density_gaussian_like(grid, center=4.0, width=0.5)

fp_history = simulate_fokker_planck(
    grid=grid,
    p0=p0,
    dt=0.0005,
    T=3000,
    drift_function=my_drift,
    D=0.5
)

plot_density(grid, fp_history[-1], title="Densità finale Fokker--Planck")

```

A.14 Estensione minimale con birth/death

La nota segnala giustamente che il modello base non include entry ed exit. Possiamo aggiungere una estensione molto semplice.

Rappresentiamo la popolazione di imprese come una lista di stati:

```
firms = [x_1, x_2, x_3, ...]
```

A ogni passo:

1. ogni impresa evolve con Euler–Maruyama;
2. ogni impresa può uscire con una certa probabilità;
3. con una certa probabilità può nascere una nuova impresa.

A.14.1 Aggiornare una popolazione di imprese

```
def update_firm_population(firms, dt, drift_function, diffusion_function,
                           birth_rate, death_rate,
                           entry_sampler, death_threshold=None):
    new_firms = []

    for x in firms:
        x_new = euler_maruyama_step(
            x=x,
            dt=dt,
            drift_function=drift_function,
            diffusion_function=diffusion_function
        )

        alive = True

        if random.random() < death_rate * dt:
            alive = False

        if death_threshold is not None and x_new < death_threshold:
            alive = False

        if alive:
            new_firms.append(x_new)

    if random.random() < birth_rate * dt:
        newborn = entry_sampler()
        new_firms.append(newborn)

    return new_firms
```

Questa è una versione molto semplice:

- al massimo una nascita per passo;
- morte casuale con intensità `death_rate`;
- morte opzionale sotto soglia.

A.14.2 Regola di ingresso

Per esempio, una nuova impresa può entrare vicino a un valore iniziale prefissato:

```
def simple_entry_sampler():
    return random.gauss(0.0, 0.5)
```

A.14.3 Simulare l'evoluzione della popolazione

```
def simulate_firm_population(firms0, dt, T,
                            drift_function, diffusion_function,
                            birth_rate, death_rate,
                            entry_sampler, death_threshold=None):

    firms = firms0[:]
    history_population_size = [len(firms)]
    history_mean_state = [statistics.mean(firms) if len(firms) > 0 else 0.0]

    for t in range(T):
        firms = update_firm_population(
            firms=firms,
            dt=dt,
            drift_function=drift_function,
            diffusion_function=diffusion_function,
            birth_rate=birth_rate,
            death_rate=death_rate,
            entry_sampler=entry_sampler,
            death_threshold=death_threshold
        )

        history_population_size.append(len(firms))

        if len(firms) > 0:
            history_mean_state.append(statistics.mean(firms))
        else:
            history_mean_state.append(0.0)

    results = {
        "final_firms": firms,
        "history_population_size": history_population_size,
        "history_mean_state": history_mean_state
    }

    return results
```

Esempio:

```
firms0 = [random.gauss(0.0, 1.0) for _ in range(30)]

population_results = simulate_firm_population(
    firms0=firms0,
    dt=0.01,
```

```

T=5000,
drift_function=my_drift,
diffusion_function=my_diffusion,
birth_rate=2.0,
death_rate=1.5,
entry_sampler=simple_entry_sampler,
death_threshold=-6.0
)

```

A.14.4 Grafici della popolazione

```

def plot_population_size(history_population_size):
    times = list(range(len(history_population_size)))

    plt.plot(times, history_population_size)
    plt.xlabel("tempo")
    plt.ylabel("numero di imprese")
    plt.title("Dinamica della popolazione di imprese")
    plt.show()

```

```

def plot_population_mean_state(history_mean_state):
    times = list(range(len(history_mean_state)))

    plt.plot(times, history_mean_state)
    plt.xlabel("tempo")
    plt.ylabel("media dello stato")
    plt.title("Media della variabile d'impresa")
    plt.show()

```

Esempio:

```

plot_population_size(population_results["history_population_size"])
plot_population_mean_state(population_results["history_mean_state"])

```

A.15 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo in questo ordine:

1. import delle librerie;
2. funzioni di base:
 - sign
 - drift
 - diffusione
3. simulazione SDE:

- euler_maruyama_step
 - simulate_sde
 - grafici
4. stazionaria empirica:
 - burn-in
 - istogramma
 - densità teorica
 5. Fokker–Planck:
 - griglia
 - normalizzazione
 - passo esplicito
 - simulazione
 6. popolazione di imprese:
 - update con birth/death
 - simulazione
 - grafici
 7. blocco finale con esempi.

Per esempio:

```
if __name__ == "__main__":
    m = 0.0
    sigma = 1.0
    alpha = 1.0
    D = 0.5

    def my_drift(x):
        return subbotin_drift(x, m=m, sigma=sigma, alpha=alpha, D=D)

    def my_diffusion(x):
        return constant_diffusion(x, D=D)

    history = simulate_sde(
        x0=4.0,
        dt=0.01,
        T=50000,
        drift_function=my_drift,
        diffusion_function=my_diffusion
    )
```



```

samples = stationary_samples_from_history(history, burn_in=10000)

def my_density(x):
    return subbotin_density(x, m=m, sigma=sigma, alpha=alpha)

plot_histogram_with_theoretical_density(
    samples=samples,
    density_function=my_density,
    bins=80,
    title="Stazionaria empirica e densità teorica"
)

```

A.16 Perché questa appendice è utile

Questa appendice è utile perché rende molto visibile una progressione metodologica centrale del modulo:

1. si parte da una SDE;
2. si simulano traiettorie;
3. si ricostruisce una distribuzione stazionaria empirica;
4. la si confronta con la densità teorica;
5. si passa, opzionalmente, a una dinamica di densità con Fokker-Planck;
6. si aggiunge infine una popolazione di imprese con nascita e morte.

Questo è esattamente il tipo di passaggio che rende il caso di studio interdisciplinare e computazionalmente ricco.

A.17 Conclusione dell'appendice

La struttura proposta qui è volutamente semplice. Chi conosce Python può implementarla quasi direttamente; chi usa altri linguaggi può leggerla come pseudocodice molto vicino a una traduzione operativa.

Il punto metodologico essenziale è che questo modulo permette di confrontare tre livelli diversi di descrizione:

1. traiettorie individuali di una SDE;
2. distribuzione stazionaria della variabile;
3. popolazione di imprese con nascita e morte.

Per questo motivo è uno dei case study più completi dell'intero corso.